

TÀI LIỆU BÀN GIAO & HƯỚNG DẪN TÍCH HỢP HỆ THỐNG

GIAI ĐOẠN 3 — PHÁT TRIỂN BACKEND AI (RAG SYSTEM)

Dự án: Second Brain App — Knowledge Base + Graph View + AI RAG Chatbot

Người báo cáo & bàn giao: Thành viên 4 (AI / RAG Developer)

□ HẠNG MỤC GIAI ĐOẠN 3 ĐÃ HOÀN TẤT 100%

1. Báo Cáo Tiến Độ & Kết Quả Giai Đoạn 3

Để nhóm trưởng có thể nắm bắt và báo cáo tiến độ chung, phần công việc của Giai đoạn 3 đã được hoàn tất với các hạng mục sau:

- ✓ **Hệ thống Ingestion:** Đã xây dựng script tự động đọc các file .md từ thư mục nội bộ (Local Folder), băm nhỏ dữ liệu (Embedding) và lưu trữ vào Cơ sở dữ liệu Vector (ChromaDB).
- ✓ **Hệ thống AI RAG Chatbot:** Đã tích hợp thành công Mô hình Ngôn ngữ Lớn (LLM Gemini), cho phép hệ thống “đọc” các tài liệu liên quan để trả lời chính xác câu hỏi của sinh viên dựa trên tài liệu của trường, cam kết không bịa đặt thông tin.
- ✓ **API Server:** Đã đóng gói toàn bộ logic thành một server Python nhỏ gọn chạy bằng FastAPI, hoạt động độc lập để không bắt ứng dụng Flutter phải gánh phần xử lý AI nặng nề. API đã được mở CORS để sẵn sàng tích hợp.

2. Đặc Tả API (API Contract) — Dành Cho Giao Diện Chatbot

Thành viên 2 (Flutter UI) sẽ sử dụng đặc tả này để kết nối giao diện Chat với Bộ não AI.

Mục	Chi tiết
Endpoint	POST http://127.0.0.1:8000/chat
Headers yêu cầu	Content-Type: application/json

Cấu trúc dữ liệu gửi lên (Request Body)

```
{  "question": "Mon MLN111 học về nhưng nội dung gì?"}
```

Cấu trúc dữ liệu trả về (JSON Response)

```
{  "answer": "Mon MLN111 bao gồm các kiến thức về Triết học Mac - Lenin...",  "sources": ["MLN111.md"]}
```

3. Hướng Dẫn & Yêu Cầu Tích Hợp Cho Các Giai Đoạn

Đối với Thành viên 1 (Data Engineer — Giai đoạn 1)

Tiến độ hiện tại: RAG Server đã đọc hiểu rất tốt định dạng file hiện tại của bạn.

- Yêu cầu duy trì: Khi tiếp tục cào (crawl) dữ liệu từ trang FLM và tạo ra hàng loạt file .md mới, vui lòng giữ nguyên cấu trúc Markdown chuẩn — bắt buộc phải có khối siêu dữ liệu (Frontmatter YAML) chứa trường id ở đầu file và chia các phần bằng thẻ Heading (##).
- Cách cập nhật dữ liệu: Mỗi khi kéo thêm dữ liệu mới vào thư mục data/knowledge_base, chỉ cần thông báo để chạy lại lệnh python ingest.py — dữ liệu sẽ tự động được nạp thêm vào ChromaDB mà không bị trùng lặp.

Đối với Thành viên 2 (Flutter UI & File Handling — Giai đoạn 2)

- Kiến trúc nền tảng: Tiếp tục dựng khung giao diện ứng dụng Desktop và sử dụng thư viện dart:io để quét đường dẫn thư mục cục bộ chứa các file .md. Nhiệm vụ là hiển thị chúng lên giao diện chia đôi màn hình (Split View): một bên là cây thư mục (File Tree), một bên dùng thư viện flutter_markdown để render nội dung text cho đẹp mắt.
- Tích hợp Chatbot: Trong giao diện Chat, sử dụng thư viện http kết hợp async/await để gửi câu hỏi của người dùng tới API /chat mô tả ở Phần 2. Sau đó xử lý chuỗi JSON Response trả về để hiển thị câu trả lời (answer) và nguồn trích dẫn (sources) lên màn hình.

Đối với Thành viên 3 (Graph Visualizer — Giai đoạn 4)

- Cơ chế hoạt động: Backend AI hoạt động hoàn toàn độc lập và chỉ tập trung vào việc đọc nội dung văn bản.
- Nhiệm vụ: Đọc trực tiếp nội dung các file .md từ thư mục cục bộ, sử dụng Regex trên Dart để tìm các từ khóa liên kết (ví dụ: [[Môn học A]], [[PRM393]]).
- Vẽ đồ thị: Dựa vào các liên kết đó, dùng các package vẽ đồ thị (như graphview, force_directed_graph hoặc CustomPaint) để trực quan hóa thành các điểm (Node — Môn học) và đường nối (Edge — Điều kiện tiên quyết) giống ứng dụng Obsidian.
- Lưu ý hiệu năng: Khi quét hàng trăm file .md để vẽ Node/Edge, cần tối ưu hiệu năng để không làm treo UI (tránh heavy loops).

4. Hướng Dẫn Khởi Chạy Hệ Thống AI Tại Local

Để các thành viên có thể tự chạy Backend AI trên máy cá nhân trong quá trình code Flutter, hãy làm theo các bước sau:

Bước 1 — Kéo code (Pull)

Kéo nhánh main từ repository Backend-RAG-System về máy.

Bước 2 — Môi trường ảo

Mở Terminal tại thư mục gốc, tạo và kích hoạt môi trường ảo.

Bước 3 — Cài đặt thư viện

Cài đặt các thư viện cần thiết: FastAPI, ChromaDB, Sentence-Transformers, Google Generative AI.

Bước 4 — Thiết lập API Key

Tạo file .env ở thư mục gốc và dán API Key của Gemini vào. Tuyệt đối không push file này lên GitHub.

Bước 5 — Khởi động Server

Chạy lệnh khởi động server. Server sẽ hoạt động ở địa chỉ `http://127.0.0.1:8000`.

```
# Bước 2: Tạo và kích hoạt môi trường ảo
python -m venv venv
.\venv\Scripts\activate      # Windows
# source venv/bin/activate   # macOS/Linux

# Bước 3: Cài đặt thư viện
pip install fastapi uvicorn chromadb sentence-transformers google-generativeai python-dotenv

# Bước 4: File .env (tạo ở thư mục gốc, KHÔNG push lên GitHub)
GEMINI_API_KEY=dan_api_key_cua_ban_vao_day

# Bước 5: Khởi động server
uvicorn main:app --reload --port 8000
# Server chạy tại: http://127.0.0.1:8000
```

📌 **Lưu ý cho cả nhóm:** Backend AI chạy hoàn toàn độc lập với ứng dụng Flutter — chỉ cần server đang chạy ở cổng 8000 là Thành viên 2 và Thành viên 3 đều có thể phát triển và test song song mà không phụ thuộc vào tiến độ của nhau.